

02: Linux Concepts, Deep Dive

Systems interviews for cloud infrastructure roles return to a small set of questions. What happens on a node when a Pod starts? Why can a container see one GPU and not another? Why was a process killed when its memory graph looked flat? How do you find the cause when a node is slow? Each answer depends on a few kernel mechanisms: tasks, virtual memory, namespaces, cgroups, the virtual filesystem, and the network stack.

This chapter explains those mechanisms in the order a container depends on them. For each one, it describes the problem the mechanism solves, how the kernel implements it, and how you can observe it on a running machine. Where a behavior is easy to misremember, a short program demonstrates it.

This chapter covers

- How Linux represents processes and threads, and what `fork`, `exec`, and `clone` do
- What a system call costs and how to measure it
- Virtual memory, page faults, and the difference between VSZ, RSS, and PSS
- How `malloc` obtains memory, and why the OOM killer runs
- Namespaces and cgroup v2, the two kernel features a container is built from
- The VFS objects behind a file descriptor, and the filesystems Kubernetes uses
- The path of a packet through a host, including netfilter, conntrack, and CNI
- A structured order for debugging an unhealthy node, including GPU nodes

Contents

About the examples	3
1. Processes and threads	4
1.1 One kernel structure for both	4
1.2 Creating processes: <code>fork</code> , <code>vfork</code> , and <code>clone</code>	4
1.3 <code>exec</code> replaces the program, not the process	9
1.4 System calls	9
1.5 Process states, zombies, and orphans	11
1.6 Signals	14
2. Memory	16
2.1 Virtual memory	16
2.2 Page faults and memory metrics	17
2.3 How <code>malloc</code> gets memory: <code>brk</code> and <code>mmap</code>	22
2.4 Overcommit and the OOM killer	24
2.5 Huge pages, NUMA, and GPU memory	25
3. Namespaces, cgroups, and containers	27
3.1 Namespaces	27
3.2 cgroup v2	29

3.3 From image to running container.....	31
3.4 GPUs in containers	32
4. Filesystems and I/O	33
4.1 The VFS and file descriptors	33
4.2 Filesystems used on Kubernetes nodes	35
4.3 Mount propagation	35
4.4 The I/O stack and pressure metrics.....	36
5. Networking	38
5.1 Sockets and TCP states	38
5.2 The path of a packet through a host.....	39
5.3 netfilter, conntrack, and kube-proxy.....	39
5.4 Network namespaces, veth pairs, and CNI	40
5.5 DNS in a Pod.....	41
6. Debugging an unhealthy node	42
6.1 A triage order	42
6.2 Scenario: Pods stuck in Pending.....	42
6.3 Scenario: Pod stuck in ContainerCreating.....	43
6.4 Scenario: Pod in CrashLoopBackOff.....	43
6.5 Scenario: a GPU node cannot allocate GPUs.....	44
6.6 Scenario: high CPU use or load without an obvious cause.....	44
7. Performance and observability tools.....	46
bpfttrace examples.....	46
When to use perf.....	47
8. Containers compared with virtual machines	48
9. Interview questions.....	49
What happens when you run docker run nginx?	49
How does a memory limit terminate a container?.....	49
A container runs uname -r and ps aux. What does it see?.....	49
Why does free show almost no free memory on a healthy server?.....	49
What is the difference between a container and a virtual machine?.....	49
What is a zombie process, and how do you remove one?	50
How do you debug a GPU node that cannot run GPU Pods?	50
What is the page cache, and should you drop it?	50
What is conntrack, and how can it cause connection failures?	50
Summary	51
Further reading.....	52

About the examples

Each program in this chapter was compiled and run, and each output block is copied from that run. The machine was an NVIDIA Jetson (aarch64, Linux 5.15) with gcc 11.4.0 and rustc 1.96 nightly. Process IDs, addresses, and timings will differ on your machine. The mechanisms do not.

To build the C programs, use gcc. The Rust versions call the kernel through the libc crate:

```
gcc -O2 -Wall -Wextra -o fork_cow fork_cow.c
cargo run --release --bin syscall_cost
```

Note: On aarch64, the instruction that enters the kernel is `svc`. On x86-64 it is `syscall`. Syscall numbers and some addresses differ between the two architectures, but every behavior described here applies to both.

1. Processes and threads

1.1 One kernel structure for both

Linux does not have separate kernel objects for processes and threads. Every unit of execution is a *task*, represented by a struct `task_struct`. A *process*, in the user-space sense, is a **thread group**: one or more tasks that share a thread group ID (TGID).

The two identifiers appear in different places:

Identifier	Kernel name	System call	In <code>/proc/<pid>/status</code>	Meaning
Process ID	TGID	<code>getpid()</code>	Tgid:	Shared by every thread in the process
Thread ID	PID	<code>gettid()</code>	Pid:	Unique to each task

The naming is inverted: what user space calls a thread ID, the kernel calls a PID. In a single-threaded process the two numbers are equal.

To list the threads of every process, add `-L` to `ps`, or press `H` in `top`:

```
ps -eLo pid,tid,ppid,stat,comm    # one line per thread
top -H -p <pid>                  # the threads of one process
ls /proc/<pid>/task/              # one directory per thread ID
```

Using one structure for both simplifies the rest of the kernel. The scheduler, signal delivery, credential checks, and cgroup accounting all operate on tasks, and a "thread" is a relationship between tasks rather than a second kind of object. The same design explains two details you can see from user space:

- `/proc/<tgid>/task/<tid>` is a directory inside the process's `/proc` entry, not a separate tree.
- `kill(2)` sends a signal to a whole thread group, and `tgkill(2)` sends it to one task in the group.

1.2 Creating processes: `fork`, `vfork`, and `clone`

Linux provides three system calls for creating a task. All three are implemented by the same kernel function; they differ in what the new task shares with its parent.

Call	What the child gets	Typical use
<code>fork()</code>	A copy of the parent's address space, file descriptor table, and signal handlers	Starting a new process
<code>vfork()</code>	The parent's address space itself; the parent is suspended until the child calls <code>exec</code> or <code>_exit</code>	Old code that predates cheap <code>fork</code> ; <code>posix_spawn</code> implementations
<code>clone(flags)</code>	Whatever <code>flags</code> selects: shared memory, shared file table, new namespaces	Threads (<code>pthread_create</code>), container runtimes

`pthread_create` calls `clone` with `CLONE_VM | CLONE_FILES | CLONE_SIGHAND | CLONE_THREAD` and related flags, so the new task shares the address space, the descriptor table, and the signal handlers, and joins the caller's thread group. A container runtime such as `runc` calls `clone` or `unshare` with flags such as `CLONE_NEWPID` and `CLONE_NEWNS` so that the new process starts in new namespaces (section 3.1).

A thread, then, is a task created with flags that share almost everything. It is not a separate concept in the kernel.

Copy-on-write

`fork()` does not copy memory when it is called. Instead, the kernel marks every writable page as read-only in both the parent's and the child's page tables, and both processes map the same physical frames. When either process writes to one of those pages, the CPU raises a protection fault. The kernel then allocates a new frame, copies that one page into it, and gives the writing process a writable mapping to the copy. This is called **copy-on-write** (COW).

Because only written pages are copied, `fork()` is fast even for a large process. It also means that you cannot compute the memory used by a parent and its child as `RSS(parent) + RSS(child)`: until a page is written, both processes count the same physical frame.

Listing 2.1 makes copy-on-write visible. The parent and the child print the same virtual address with different values.

Listing 2.1 fork_cow.c: the child writes to a private copy of the page

```

/* fork_cow.c: the child gets a private copy of the parent's memory. */
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void)
{
    int value = 42;                /* on the parent's stack */
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        value = 99;                /* writes to a private copy */
        printf("child : pid=%d value=%d address=%p\n",
               (int) getpid(), value, (void *)&value);
        fflush(stdout);            /* flush before _exit, or this line is lost */
        _exit(0);                 /* exit() would flush too, and run atexit handlers */
    }

    waitpid(pid, NULL, 0);
    printf("parent: pid=%d value=%d address=%p\n",
           (int) getpid(), value, (void *)&value);
    return 0;
}

```

```

$ gcc -O2 -o fork_cow fork_cow.c && ./fork_cow
child : pid=1939075 value=99 address=0xfffffeb225404
parent: pid=1939074 value=42 address=0xfffffeb225404

```

Both lines show the address 0xfffffeb225404, but the values differ. The address is virtual. After the child's write, the same virtual address maps to a different physical frame in each process. In `/proc/<pid>/smaps`, the copied page appears as a private dirty page in each process.

Warning: buffered output and `_exit`

Listing 2.1 calls `fflush(stdout)` before `_exit(0)` for a specific reason. When standard output is a pipe or a file, the C library buffers it in blocks rather than lines. `exit(3)` flushes that buffer; `_exit(2)` does not. A child that prints and then calls `_exit` loses its output, and nothing reports an error. Listing 2.2 shows the failure.

Listing 2.2 `lost.c`: output disappears when the buffer is never flushed

```

/* lost.c: the child's output disappears. */
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("child\n"); /* sits in the buffer */
    _exit(0);          /* does not flush it */
}

```

```

$ gcc -O2 -o lost lost.c && echo "captured: [$(./lost)]"
captured: []

```

The opposite failure also occurs. If a process has unflushed output when it calls `fork()`, the buffer is copied into the child. When both processes later call `exit()`, both flush the same bytes, and the output appears twice. Flush standard output before calling `fork()`, and call `_exit()` rather than `exit()` in a child that does not call `exec`.

Copy-on-write in Rust

Rust calls `fork(2)` through the `libc` crate. The important constraint is on the child: between `fork()` and `exec()`, POSIX allows only *async-signal-safe* functions. Most of `std::io`, including `println!`, can take locks and use buffers that were copied from the parent, so it is not safe to call there. Listing 2.3 writes with `write(2)` directly.

Listing 2.3 fork_cow.rs: the same demonstration through libc

```

/// fork_cow.rs
use libc::{fork, getpid, waitpid};

/// `write(2)` directly: after `fork()` only async-signal-safe calls belong in the
/// child. `println!` is not one of them.
fn raw_print(text: &str) {
    // SAFETY: `text` is valid for `text.len()` bytes for the duration of the call.
    unsafe { libc::write(1, text.as_ptr() as *const libc::c_void, text.len()) };
}

fn main() {
    let mut value: i32 = 42; // on the parent's stack

    // SAFETY: single-threaded, and the child performs one write() then _exit().
    let pid = unsafe { fork() };

    if pid < 0 {
        eprintln!("fork failed");
        std::process::exit(1);
    }

    if pid == 0 {
        value = 99; // writes into the child's private copy of the page
        raw_print(&format!(
            "child : pid={} value={} address={:p}\n",
            unsafe { getpid() },
            value,
            &value as *const i32
        ));
        // SAFETY: _exit skips destructors and the stdio flush, which is the point.
        unsafe { libc::_exit(0) };
    }

    // SAFETY: `pid` is the child created above.
    unsafe { waitpid(pid, std::ptr::null_mut(), 0) };
    println!(
        "parent: pid={} value={} address={:p}",
        unsafe { getpid() },
        value,
        &value as *const i32
    );
}

```

```

$ cargo run --release --bin fork_cow
child : pid=1938714 value=99 address=0xffffd94eee74
parent: pid=1938713 value=42 address=0xffffd94eee74

```

Note: `format!` in the child allocates, and allocation is not async-signal-safe either. The listing is acceptable only because the process is single-threaded, so no other thread can hold the allocator's lock at the moment of the fork. In a multithreaded program, a thread that holds a lock during `fork()` does not exist in the child, and the child deadlocks the first time it takes that lock. In production code, use `std::process::Command`, which runs `exec` immediately after `fork` (or uses `posix_spawn`), so no Rust code runs in between.

1.3 `exec` replaces the program, not the process

`execve(path, argv, envp)` does not create a process. It discards the calling process's address space, loads the new program, and starts it at its entry point. The process ID stays the same.

Some state survives `exec`: the process ID, the parent, the current directory, resource limits, and open file descriptors. A descriptor is closed during `exec` only if its `close-on-exec` flag (`FD_CLOEXEC`) is set. Servers open sockets and files with `O_CLOEXEC` (or `SOCK_CLOEXEC`) so that a program they launch does not inherit descriptors it should not have. Rust's standard library sets the flag on every descriptor it opens.

A shell runs a command in two steps:

```
bash (pid 100)
└─ fork()                child bash, pid 101
   └─ execve("/usr/bin/curl")  pid 101 is now curl
```

Splitting creation (`fork`) from loading (`exec`) gives the child a window in which it is still running the parent's code. The shell uses that window to set up redirections, pipes, and process groups by adjusting file descriptors before the new program starts.

1.4 System calls

A user-space program cannot access hardware or kernel memory directly. To read a file, send a packet, or create a process, it asks the kernel through a *system call*. A call such as `read()` proceeds as follows:

1. The C library wrapper places the syscall number and arguments in registers.
2. It executes the trap instruction: `syscall` on x86-64 or `svc` on aarch64.
3. The CPU switches to kernel mode and jumps to the kernel's entry point.
4. The kernel validates the arguments, copies data from user memory where needed, performs the operation on the task's kernel stack, and copies results back.
5. The CPU returns to user mode with the result in a register. On failure, the C library wrapper stores the error code in `errno` and returns `-1`.

If the operation cannot finish immediately, for example a `read()` on a socket with no data, the kernel puts the task to sleep and the scheduler runs another task.

To see the system calls a process makes, use `strace`:

```
strace -f -tt -p <pid>      # follow threads and children, with timestamps
strace -c ./program         # count calls and time spent in each
```

What a system call costs

Even a system call that does almost no work has a fixed cost: the CPU saves user state, switches stacks, runs the kernel entry and exit paths, and restores state. Mitigations for CPU vulnerabilities such as Spectre and Meltdown add to that path on affected processors. Listing 2.4 measures the fixed cost with `getpid()`, one of the cheapest calls available.

Listing 2.4 `syscall_cost.rs`: timing one million `getpid()` calls

```
///! syscall_cost.rs
use std::hint::black_box;
use std::time::Instant;

const ITERATIONS: u32 = 1_000_000;

fn main() {
    // Warm up: the first calls pay for page faults and lazy symbol resolution.
    for _ in 0..10_000 {
        black_box(unsafe { libc::getpid() });
    }

    let start = Instant::now();
    for _ in 0..ITERATIONS {
        // SAFETY: getpid takes no arguments and cannot fail.
        black_box(unsafe { libc::getpid() });
    }
    let elapsed = start.elapsed();

    println!("{}", getpid() calls in {:?}", ITERATIONS, elapsed);
    println!(
        "{:.1} ns per call",
        elapsed.as_nanos() as f64 / f64::from(ITERATIONS)
    );
}
```

```
$ cargo run --release --bin syscall_cost
1000000 getpid() calls in 229.03379ms
229.0 ns per call
```

The result, about 229 nanoseconds per call, applies to this machine, kernel, and CPU frequency setting. The repository's `src/bin/syscall_overhead.rs` measured 207.6 ns for the same call on the same machine at a different time. When you quote a measurement, state the machine it came from and expect variation of this size between runs.

The fixed cost has practical consequences. A program that calls `read()` once per byte spends more time entering and leaving the kernel than copying data. The common techniques for reducing syscall overhead all reduce the number of crossings:

Technique	How it reduces crossings
Buffered I/O (BufReader, stdio)	Many small reads become one large <code>read()</code>
<code>readv / writev</code>	Several buffers in one call
<code>sendmmsg / recvmmsg</code>	Several datagrams in one call
<code>io_uring</code>	Requests and completions pass through shared ring buffers
<code>mmap</code>	File contents are accessed as memory, with no <code>read()</code> per access
vDSO	Calls such as <code>clock_gettime</code> run in user space with no trap
Kernel-bypass networking (DPDK, RDMA)	The data path does not enter the kernel

When a program is slow and you suspect system calls, `strace -c` shows how many calls it made and how long they took. `strace` itself slows the traced program considerably; for lower overhead, use `perf trace`.

A stable interface

The syscall numbers, the argument conventions, and the `-1-plus-errno` error convention form the kernel's application binary interface (ABI) with user space. Linux kernel policy is that a change must not break existing user-space programs, so new behavior is added as new system calls or new flags rather than by changing existing ones. A statically linked binary built many years ago still runs on a current kernel for this reason. The same approach applies when you design a public API: add fields, flags, or versions instead of changing the meaning of what already exists.

1.5 Process states, zombies, and orphans

The STAT column in `ps` shows each task's state:

State	Name	Meaning
R	Running	Running on a CPU or waiting in the run queue
S	Interruptible sleep	Waiting for an event; a signal wakes it
D	Uninterruptible sleep	Waiting inside the kernel, usually for I/O; signals are deferred
T	Stopped	Stopped by <code>SIGSTOP</code> or a debugger
Z	Zombie	Exited, but the parent has not collected the exit status
I	Idle	An idle kernel thread

Zombies. When a process exits, the kernel frees its memory and closes its files but keeps a small part of its task structure, so that the parent can read the exit status with `wait()` or `waitpid()`. Until the parent does, the process is a zombie. A zombie uses almost no memory, but it still holds a process ID, and process IDs are limited by `/proc/sys/kernel/pid_max` and by the cgroup's `pids.max`.

Orphans. If a parent exits before its children, the children are reparented to PID 1 (or to the nearest ancestor marked as a subreaper), which is expected to reap them when they exit.

Uninterruptible sleep. A task in state D is waiting inside the kernel, usually on disk or network storage, and does not act on signals until the operation completes. `kill -9` has no effect until then. Many tasks in state D usually point to a storage or network filesystem problem, such as an unresponsive NFS server. On Linux, tasks in state D also count toward the load average, so a node can report a high load average while its CPUs are idle.

Observing a zombie

Field 3 of `/proc/<pid>/stat` is the same state letter that `ps` prints. Listing 2.5 creates a child, lets it exit without reaping it, and reads that field before and after calling `waitpid`.

Listing 2.5 zombie.c: an exited child remains until it is reaped

```

/* zombie.c: an exited child stays in the process table until it is reaped. */
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

static void show_state(const char *label, pid_t pid)
{
    char path[64];
    FILE *f;
    int first;
    char comm[64];
    char state;

    snprintf(path, sizeof path, "/proc/%d/stat", (int)pid);
    f = fopen(path, "r");
    if (f == NULL) {
        printf("%s: /proc/%d/stat is gone\n", label, (int)pid);
        return;
    }
    /* field 1 is the pid, field 2 the command in parentheses, field 3 the state */
    if (fscanf(f, "%d %63s %c", &first, comm, &state) == 3)
        printf("%s: state '%c'\n", label, state);
    fclose(f);
}

int main(void)
{
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0)
        _exit(0); /* the child is gone in microseconds */

    printf("parent %d, child %d has exited\n", (int)getpid(), (int)pid);
    sleep(1); /* let the child exit first */
    show_state("exited, not reaped", pid);
    show_state("still not reaped", pid);

    waitpid(pid, NULL, 0); /* only now is the child reaped */
    show_state("after waitpid", pid);
    return 0;
}

```

```

$ gcc -O2 -o zombie zombie.c && ./zombie
parent 1939371, child 1939372 has exited
exited, not reaped: state 'Z'
still not reaped: state 'Z'
after waitpid: /proc/1939372/stat is gone

```

The child has no code left to run, but its entry remains, in state Z, until the parent calls `waitpid`. After that call, the kernel releases the entry and the `/proc` directory disappears.

The `sleep(1)` is there because `fork` does not decide which process runs first. Without it, the parent can read the file before the child has exited and see R or S. When you check for zombies on a real system, read the state more than once.

Note: In a container, the process that runs as PID 1 inherits every orphan in the container's PID namespace. Most applications do not call `wait` for children they did not create, so a container whose PID 1 is an ordinary application can accumulate zombies. Run a minimal init process such as `tini` (Docker's `--init` flag), or set `shareProcessNamespace: true` in the Pod spec so that the pause container becomes PID 1 and reaps orphans.

1.6 Signals

A signal is an asynchronous notification delivered to a process or thread. For each signal, a process can accept the default action, ignore the signal, or install a handler, with two exceptions: `SIGKILL` and `SIGSTOP` cannot be caught, blocked, or ignored.

Signal	Number	Default action	Common use
<code>SIGHUP</code>	1	Terminate	Terminal closed; many daemons reload configuration
<code>SIGINT</code>	2	Terminate	Ctrl+C in a terminal
<code>SIGKILL</code>	9	Terminate	Forced kill; cannot be handled
<code>SIGSEGV</code>	11	Terminate and dump core	Invalid memory access
<code>SIGTERM</code>	15	Terminate	Polite request to shut down
<code>SIGCHLD</code>	17	Ignore	A child stopped or exited
<code>SIGCONT</code>	18	Continue	Resume a stopped process
<code>SIGSTOP</code>	19	Stop	Pause a process; cannot be handled

The numbers shown are for x86-64 and aarch64. A few signals have different numbers on other architectures, so scripts should use names such as `kill -TERM` rather than numbers.

Pod termination in Kubernetes. When a Pod is deleted, the kubelet runs any `preStop` hook, then sends `SIGTERM` to PID 1 of each container. If the container is still running when `terminationGracePeriodSeconds` (30 seconds by default) expires, the kubelet sends `SIGKILL`. The signal goes only to PID 1, so an application started by a shell script (`sh -c "myserver"`) may never receive `SIGTERM` unless the script uses `exec` or the image uses an init process that forwards signals.

Useful commands:

```
ps -eo pid,tid,ppid,stat,comm --sort=-%cpu | head    # processes by CPU use
pstree -ap                                           # the process tree, with arguments
pgrep -a python                                     # find processes by name
kill -TERM 1234                                     # ask process 1234 to exit
kill -KILL 1234                                     # force it to exit
renice -n 5 -p 1234                                 # lower its scheduling priority
grep -E 'Sig(Blk|Ign|Cgt)' /proc/1234/status        # blocked, ignored, and caught signals
```

2. Memory

2.1 Virtual memory

Every process has its own *virtual address space*. The kernel describes it with a `struct mm_struct` and a set of page tables. The CPU's memory management unit translates each virtual address to a physical address by walking those tables, and it caches recent translations in the translation lookaside buffer (TLB).

Because each process has its own tables, virtual memory provides several features with one mechanism:

Feature	How the page tables provide it
Isolation	A process's tables contain no entries for another process's frames
Lazy allocation	An entry is filled in only when the page is first touched
Shared libraries and page cache	Entries in many processes point at the same frame
Copy-on-write	Shared entries are marked read-only until one process writes
Overcommit	Address space can be promised before frames exist to back it

A typical 64-bit user address space is laid out as follows, from low addresses to high:

```

low addresses
├─ program text (machine code)
├─ data and BSS (initialized and zero-initialized globals)
├─ heap (grows upward; extended with brk)
|
├─ memory-mapped region (shared libraries, mmap files, thread stacks, large malloc blocks)
|
├─ main thread stack (grows downward)
├─ vDSO and vvar (kernel-provided pages for fast system calls)
high addresses

```

Run `cat /proc/self/maps` to see the actual layout of a process.

How an address is translated

With 4 KiB pages and 48-bit virtual addresses, the kernel uses four levels of page tables. The 48 bits of an address are divided into four 9-bit table indexes and a 12-bit offset within the page:

```

47      39 38      30 29      21 20      12 11      0
+-----+-----+-----+-----+
| PGD 9 bit| PUD 9 bit| PMD 9 bit| PTE 9 bit| offset 12 |
+-----+-----+-----+-----+

```

The MMU uses the first index to select an entry in the top-level table (PGD). That entry points to a PUD table, whose selected entry points to a PMD table, whose entry points to a PTE table. The PTE contains the physical frame number. The 12-bit offset is added to that frame's address unchanged; 2^{12} bytes is 4 KiB, the page size.

A few numbers follow directly from this layout:

- Each table has $2^9 = 512$ entries. At 8 bytes per entry, a table fills exactly one 4 KiB page.
- One PTE table maps 512 pages, or 2 MiB of address space.
- Tables for unused regions are never allocated, so a process that uses a few megabytes needs only a few tables, even though its address space spans more than 100 TiB.

A translation that is not in the TLB requires up to four memory reads before the actual access. TLB misses are therefore a measurable cost for programs with large, scattered working sets. You can count them with `perf stat -e dTLB-load-misses`. Huge pages (section 2.5) reduce the number of misses.

2.2 Page faults and memory metrics

Memory is managed in pages, usually 4 KiB, with 2 MiB and 1 GiB huge pages also available. When a program maps memory, for example with a large `malloc`, the kernel records the range of virtual addresses but does not allocate physical frames. The first access to each page causes a **page fault**, and the kernel handles it by allocating a frame and filling in the page table entry.

The kernel classifies faults by what it has to do:

Fault	What the kernel does	Examples
Minor	Maps a frame without reading from storage	First touch of anonymous memory (a zero-filled frame); a file page already in the page cache; a copy-on-write copy
Major	Reads the page from storage before mapping it	A file page not in the page cache; a page in swap
Invalid	Sends SIGSEGV to the process	An address with no mapping, or a write to a read-only mapping that is not copy-on-write

Several metrics describe how much memory a process uses. They measure different things, and confusing them is a common interview mistake:

Metric	Measures	Where to find it
VSZ	All mapped virtual address space, whether or not it is backed by frames	<code>ps -o vsz, VmSize</code> in <code>/proc/<pid>/status</code>
RSS	Frames currently mapped into the process, including shared ones	<code>ps -o rss, VmRSS</code>
PSS	RSS with each shared page divided by the number of processes that map it	<code>/proc/<pid>/smaps_rollup</code>
USS	Frames mapped only by this process	<code>smem</code> , or the <code>Private_*</code> lines of <code>smaps_rollup</code>

VmRSS is further split into `RssAnon` (heap, stack, and other anonymous memory), `RssFile` (mapped files and libraries), and `RssShmem` (shared memory). For a process's own memory use, `RssAnon` is usually the most informative value.

The **page cache** holds recently used file data in otherwise unused RAM. It is normal for free to show most memory as used by `buff/cache`. The kernel reclaims cache when applications need memory, so the value to watch is `available`, not `free`.

```
grep -E 'VmSize|VmRSS|RssAnon|RssFile' /proc/<pid>/status
cat /proc/<pid>/smaps_rollup
free -h
vmstat 1 5          # si/so columns show swap activity
```

Counting page faults

The kernel counts faults for each process. `getrusage(RUSAGE_SELF)` returns them in `ru_minflt` and `ru_majflt`. The same counters appear as fields 10 and 12 of `/proc/<pid>/stat` and in `ps -o min_flt,maj_flt`. Listing 2.6 maps 64 MiB of anonymous memory, writes to all of it twice, and prints the number of faults for each pass.

Listing 2.6 faults.c: counting page faults with getrusage

```

/* faults.c: the kernel counts the page faults a process takes. */
#define _GNU_SOURCE
#include <stdio.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/resource.h>

static void delta(const char *label, struct rusage *before, struct rusage *after)
{
    printf("%-13s minor +%ld major +%ld\n", label,
           after->ru_minflt - before->ru_minflt,
           after->ru_majflt - before->ru_majflt);
}

int main(void)
{
    const size_t len = 64u * 1024 * 1024; /* 64 MiB */
    struct rusage before, after;
    char *p;

    p = mmap(NULL, len, PROT_READ | PROT_WRITE,
             MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (p == MAP_FAILED) {
        perror("mmap");
        return 1;
    }

    getrusage(RUSAGE_SELF, &before);
    memset(p, 1, len); /* first touch of every page */
    getrusage(RUSAGE_SELF, &after);
    delta("first touch", &before, &after);

    getrusage(RUSAGE_SELF, &before);
    memset(p, 2, len); /* already resident */
    getrusage(RUSAGE_SELF, &after);
    delta("second write", &before, &after);

    printf("mapped %zu bytes = %zu pages of %d\n", len, len / 4096, 4096);
    munmap(p, len);
    return 0;
}

```

```

$ gcc -O2 -o faults faults.c && ./faults
first touch  minor +16384 major +0
second write minor +0 major +0
mapped 67108864 bytes = 16384 pages of 4096

```

64 MiB divided by 4 KiB is 16,384 pages, and the first pass took exactly 16,384 minor faults: one per page. The second pass took none, because every page already had a frame and a page table entry. There were no major faults, because anonymous memory has no backing file to read.

A file mapping behaves differently. The first touch of a page is a minor fault if the page is already in the page cache and a major fault if it is not, so the same program run against a file splits its faults between the two counters depending on the state of the cache. In production, a sustained rate of major faults is a sign that the working set no longer fits in memory.

Listing 2.7 reads the same counters from Rust.

Listing 2.7 faults.rs: the same measurement through libc

```

/// faults.rs
use libc::{getrusage, mmap, rusage, MAP_ANONYMOUS, MAP_FAILED, MAP_PRIVATE, PROT_READ, PROT_WRITE,
RUSAGE_SELF};

const LEN: usize = 64 * 1024 * 1024; // 64 MiB

fn usage() -> rusage {
    let mut u: rusage = unsafe { std::mem::zeroed() };
    // SAFETY: RUSAGE_SELF writes a rusage into a valid stack slot.
    unsafe { getrusage(RUSAGE_SELF, &mut u) };
    u
}

fn main() {
    // SAFETY: an anonymous private mapping; the result is checked below.
    let base = unsafe {
        mmap(
            std::ptr::null_mut(),
            LEN,
            PROT_READ | PROT_WRITE,
            MAP_PRIVATE | MAP_ANONYMOUS,
            -1,
            0,
        )
    };
    if base == MAP_FAILED {
        eprintln!("mmap failed");
        std::process::exit(1);
    }

    let before = usage();
    // SAFETY: the mapping is LEN bytes and writable.
    unsafe { std::ptr::write_bytes(base as *mut u8, 1, LEN) };
    let after = usage();
    println!(
        "first touch : minor +{} major +{}",
        after.ru_minflt - before.ru_minflt,
        after.ru_majflt - before.ru_majflt
    );

    let before = usage();
    // SAFETY: same mapping, already resident.
    unsafe { std::ptr::write_bytes(base as *mut u8, 2, LEN) };
    let after = usage();
    println!(
        "second write: minor +{} major +{}",
        after.ru_minflt - before.ru_minflt,
        after.ru_majflt - before.ru_majflt
    );
    println!("bytes: {} = {} pages", LEN, LEN / 4096);
}

```

```
$ cargo run --release --bin faults
first touch : minor +16384 major +0
second write: minor +0 major +0
bytes: 67108864 = 16384 pages
```

Both programs call `mmap` directly instead of `malloc`. This keeps the memory allocator out of the measurement, so the counts reflect only the kernel's behavior. The next section adds the allocator back.

2.3 How `malloc` gets memory: `brk` and `mmap`

`malloc` is a user-space library function, not a system call. The glibc allocator obtains memory from the kernel in two ways and subdivides it for the program:

- **The heap, extended with `brk`.** Small allocations come from a contiguous region after the program's data segment. The allocator moves the end of that region, the *program break*, with the `brk` system call.
- **Separate `mmap` mappings.** Allocations at or above `M_MMAP_THRESHOLD`, which starts at 128 KiB, get their own anonymous mapping. `free` returns such a mapping to the kernel immediately with `munmap`.

Multithreaded programs also use additional *arenas*, which are regions obtained with `mmap`, so that threads do not contend for a single heap lock.

Listing 2.8 prints the program break and the number of lines in `/proc/self/maps` (one line per mapping) around a small and a large allocation.

Listing 2.8 heap.c: small blocks come from brk, large blocks from mmap

```

/* heap.c: glibc grows the brk area for small blocks and calls mmap for large ones. */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

static int mappings(void)
{
    FILE *f = fopen("/proc/self/maps", "r");
    int lines = 0;
    char buf[512];

    if (f == NULL)
        return -1;
    while (fgets(buf, sizeof buf, f) != NULL)
        lines++;
    fclose(f);
    return lines;
}

int main(void)
{
    void *small;
    void *large;

    printf("start    brk=%p mappings=%d\n", sbrk(0), mappings());

    small = malloc(64 * 1024);
    printf("64 KiB    %p brk=%p mappings=%d\n", small, sbrk(0), mappings());

    large = malloc(8 * 1024 * 1024);
    printf("8 MiB     %p brk=%p mappings=%d\n", large, sbrk(0), mappings());

    free(large);
    free(small);
    printf("freed     brk=%p mappings=%d\n", sbrk(0), mappings());
    return 0;
}

```

```

$ gcc -O2 -o heap heap.c && ./heap
start    brk=0xaaaaef254000 mappings=16
64 KiB   0xaaaaef2558a0 brk=0xaaaaef275000 mappings=16
8 MiB    0xffff8143f010 brk=0xaaaaef275000 mappings=17
freed     brk=0xaaaaef275000 mappings=16

```

Read the output one line at a time:

1. **64 KiB.** The block's address is just above the starting break, the break moved up by 132 KiB (0x21000 bytes), and the number of mappings did not change. The block came from the heap, and the allocator extended the heap by more than it needed so that later small allocations do not each require a system call.

2. **8 MiB.** The break did not move, the number of mappings increased from 16 to 17, and the address is in the high part of the address space where the kernel places `mmap` regions. The block received its own mapping.
3. **After both free calls.** The 8 MiB mapping is gone, so that memory was returned to the kernel. The break has not moved back, so the heap memory was kept for reuse.

Two production consequences follow from this behavior:

- **Freed heap memory often stays in the process.** The allocator returns heap memory only when there is a large free region at the top of the heap. Freed blocks in the middle of the heap remain part of the process. A service that allocates many small objects during a traffic spike can keep a high RSS after the spike ends, without a leak. `malloc_trim(0)` asks glibc to release what it can.
- **The `mmap` threshold changes at run time.** When a program frees a block that was served by `mmap`, glibc raises the threshold to that block's size (up to 32 MiB on 64-bit systems). After Listing 2.8 frees its 8 MiB block, a second 8 MiB request would come from the heap instead. Setting `M_MMAP_THRESHOLD` with `mallopt` or the `MALLOC_MMAP_THRESHOLD_` environment variable disables this adjustment.

Because memory is virtual until touched, allocating a large buffer is cheap, but writing to all of it creates real memory use and can trigger the OOM killer if it exceeds a host or cgroup limit.

2.4 Overcommit and the OOM killer

Linux can let an allocation succeed even when the system does not have enough RAM and swap to back it. This is called **overcommit**, and the policy is set by the `vm.overcommit_memory` sysctl:

Mode	Policy
0 (default)	Heuristic: refuse only allocations that obviously cannot be satisfied
1	Always allow
2	Never overcommit beyond swap plus a fraction of RAM (<code>vm.overcommit_ratio</code> , 50% by default)

Linux overcommits by default for two reasons. Many programs reserve much more address space than they use, such as thread stacks and sparse arrays. And `fork` would fail for large processes if the kernel had to reserve memory for every page the child might eventually copy.

The consequence is that a successful `malloc` does not guarantee that memory will be available when the program writes to it. If the system runs out of memory, the failure occurs later, during a page fault, and the kernel's response is the **OOM killer**. It selects a process by its `oom_score`, which is based mainly on the process's memory use, adjusted

by `/proc/<pid>/oom_score_adj` (from -1000, never kill, to 1000). The process that is killed is not necessarily the one that made the last allocation.

OOM kills in containers

In a container, the memory limit is enforced by the cgroup v2 memory controller (section 3.2). When a cgroup's usage reaches `memory.max`, the kernel first tries to reclaim memory charged to that cgroup, such as page cache. If it cannot reclaim enough, it runs the OOM killer *within that cgroup only*. Other containers on the node are not affected.

When this happens:

- `dmesg` shows a message such as `Memory cgroup out of memory: Killed process ....`
- The cgroup's `memory.events` file increments its `oom_kill` counter.
- Kubernetes reports the container's last state as `OOMKilled` with exit code 137 (128 + 9, for SIGKILL).

The kubelet also sets `oom_score_adj` according to each Pod's quality-of-service class, so that the host OOM killer, if it runs, prefers to kill less important Pods:

QoS class	Condition	oom_score_adj
Guaranteed	Every container has equal CPU and memory requests and limits	-997
Burstable	At least one request or limit set, but not Guaranteed	2 to 999, lower for larger memory requests
BestEffort	No requests or limits	1000

Note: A container can be OOM-killed while its RSS graph looks flat. Two causes are common. First, monitoring samples every few seconds and misses a short spike. Second, the cgroup also charges page cache and kernel memory (such as socket buffers) to the container, and those do not appear in the process's RSS. Compare `memory.current` and `memory.stat` with the limit instead.

2.5 Huge pages, NUMA, and GPU memory

Huge pages. A 2 MiB huge page is mapped by a single PMD entry instead of 512 PTE entries, so one TLB entry covers 512 times as much memory. Workloads with large memory footprints, such as databases, JVMs, and HPC applications, see fewer TLB misses. Linux provides huge pages in two ways:

- **Transparent huge pages (THP)** are used automatically for suitable anonymous memory. The setting is in `/sys/kernel/mm/transparent_hugepage/enabled`. THP can cause latency spikes when the kernel compacts memory to create huge pages, so some databases recommend `madvise` mode.

- **Preallocated huge pages** (`hugetlbfs`) are reserved in advance. Kubernetes exposes them as the resources `hugepages-2Mi` and `hugepages-1Gi`.

NUMA. On a server with several CPU sockets, each socket has its own memory, and access to local memory is faster than access to another socket's memory. PCIe devices, including GPUs and network cards, are also attached to a specific socket. Data copied between a GPU and memory on the other socket crosses the inter-socket link and is slower. `numactl --hardware` shows nodes and distances, and `nvidia-smi topo -m` shows the placement of GPUs relative to CPUs and NICs. The Kubernetes Topology Manager aligns CPU, memory, and device allocation on one NUMA node when configured to do so.

GPU memory. Several points distinguish GPU memory from host memory:

- GPU memory is separate from host RAM. `nvidia-smi` reports it, and it does not appear in a process's RSS.
- CUDA *pinned* (page-locked) host memory cannot be swapped out or moved, so the GPU can copy to and from it directly with DMA. Pinned memory makes transfers faster, and it reduces the memory available to the rest of the system.
- GPUDirect RDMA and GPUDirect Storage let a NIC or an NVMe device transfer data directly to GPU memory without a copy through host memory.

3. Namespaces, cgroups, and containers

A Linux container is not a kernel object. It is an ordinary process tree that the kernel runs with two kinds of restriction: **namespaces** limit what the processes can see, and **cgroups** limit what they can use. This section covers both and then follows a container from image to running process.

3.1 Namespaces

A namespace gives a group of processes their own instance of a global resource. Linux provides eight types:

Namespace	clone flag	What it isolates
Mount	CLONE_NEWNS	The set of mount points, and therefore the filesystem tree
PID	CLONE_NEWPID	Process IDs; the first process in the namespace is PID 1
Network	CLONE_NEWNET	Interfaces, IP addresses, routes, firewall rules, and ports
UTS	CLONE_NEWUTS	Hostname and NIS domain name
IPC	CLONE_NEWIPC	System V IPC objects and POSIX message queues
User	CLONE_NEWUSER	User and group IDs, and capabilities
Cgroup	CLONE_NEWCGROUP	The visible root of the cgroup hierarchy
Time	CLONE_NEWTIME	Offsets for CLOCK_MONOTONIC and CLOCK_BOOTTIME

Three system calls work with namespaces:

- `clone(flags)` creates a new task in new namespaces.
- `unshare(flags)` moves the calling process into new namespaces.
- `setns(fd, type)` joins an existing namespace, identified by a file in `/proc/<pid>/ns/`. `nsenter` and `kubectl exec` use it.

A process in a new PID namespace sees itself as PID 1, and it sees only the processes in its own namespace. The host sees the same process under a different, host-wide PID. Run `ls -l /proc/<pid>/ns/` to see which namespaces a process belongs to: two processes with the same inode number for a type share that namespace.

How namespaces are implemented

Namespaces are a lookup mechanism, not a copy of the system. The kernel keeps one object for each namespace instance, such as a `struct uts_namespace` or a `struct net`, and each task points at one instance of each type. When a system call resolves a name, such as a hostname, a PID, a mount path, or a port, it looks the name up in the namespace that the calling task points at. Creating a namespace allocates a new object and changes the task's pointer.

This design has two consequences that interviewers often ask about:

- **Containers start quickly.** Creating namespaces takes microseconds, and there is no guest kernel to boot.
- **All containers share one kernel.** A kernel vulnerability that a process can reach from inside a namespace affects the whole host. For stronger isolation, use a sandboxed runtime such as gVisor or Kata Containers, or a virtual machine (section 8).

Creating a UTS namespace

The UTS namespace holds only the hostname and the NIS domain name, so it is the simplest namespace to demonstrate. Listing 2.9 moves the process into a new UTS namespace and changes the hostname there.

Listing 2.9 ns.c: changing the hostname inside a new UTS namespace

```
/* ns.c: the UTS namespace holds the hostname, so a process can have its own. */
#define _GNU_SOURCE
#include <errno.h>
#include <sched.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main(void)
{
    char before[256];
    char after[256];

    if (gethostname(before, sizeof before) != 0) {
        perror("gethostname");
        return 1;
    }

    if (unshare(CLONE_NEWUTS) != 0) {
        printf("unshare(CLONE_NEWUTS) = -1 errno=%d (%s)\n", errno, strerror(errno));
        printf("this needs CAP_SYS_ADMIN, so try:\n");
        printf("  unshare --user --map-root-user --uts ./ns\n");
        return 1;
    }

    if (sethostname("in-a-namespace", 14) != 0) {
        perror("sethostname");
        return 1;
    }
    gethostname(after, sizeof after);

    printf("hostname before: %s\n", before);
    printf("hostname after : %s\n", after);
    return 0;
}
```

Run as an ordinary user, the program fails:

```
$ gcc -O2 -o ns ns.c && ./ns
unshare(CLONE_NEWUTS) = -1 errno=1 (Operation not permitted)
this needs CAP_SYS_ADMIN, so try:
  unshare --user --map-root-user --uts ./ns
```

Creating most namespaces requires the `CAP_SYS_ADMIN` capability. The exception is the user namespace, which an unprivileged user can create. Inside a new user namespace, the user's ID can be mapped to root, and that root user holds the capabilities needed to create other namespaces, but only within the new user namespace. The `unshare` command does this:

```
$ unshare --user --map-root-user --uts ./ns
hostname before: yahboom
hostname after : in-a-namespace
```

The hostname changed inside the namespace, and the host's hostname did not. Rootless container runtimes, such as rootless Podman, rely on the same mechanism: the user namespace grants capabilities that apply only to resources the namespace owns.

3.2 cgroup v2

Control groups (cgroups) organize processes into a hierarchy and apply resource accounting and limits to each group. Current distributions and Kubernetes use cgroup v2, which exposes a single tree under `/sys/fs/cgroup`. Each directory is a cgroup, and each file in it is a setting or a statistic.

The most important interface files are:

File	Purpose
cgroup.procs	Process IDs in this cgroup; write a PID to move a process here
cgroup.subtree_control	Controllers enabled for child cgroups
cpu.max	CPU quota as \$MAX \$PERIOD in microseconds; 50000 100000 allows half a CPU
cpu.weight	Relative CPU share when CPUs are contended (1 to 10000, default 100)
cpu.stat	CPU usage, and nr_throttled and throttled_usec for quota enforcement
memory.max	Hard memory limit; exceeding it after reclaim causes an OOM kill
memory.high	Soft limit; above it, the kernel throttles allocation and reclaims aggressively
memory.current	Current memory charged to the cgroup, including page cache
memory.events	Counters, including high, max, and oom_kill
pids.max	Maximum number of tasks, which limits fork bombs
io.max	Per-device limits on bytes and operations per second
cpuset.cpus, cpuset.mems	CPUs and NUMA nodes the cgroup can use

Three rules of cgroup v2 explain behavior you will see on a node:

- **No internal processes.** A non-root cgroup that distributes a controller to child cgroups cannot also contain processes itself.
- **Children inherit their parent's cgroup.** A forked process starts in its parent's cgroup and stays there unless it is moved.
- **Memory is charged to the cgroup that first touched it.** This includes page cache. A container that reads a large file is charged for the cached pages, which the kernel can reclaim when the cgroup reaches its limit.

How Kubernetes maps resources to cgroups

The kubelet creates a cgroup for each Pod and each container, and it translates the resource fields in the Pod spec into cgroup settings:

Pod spec field	cgroup v2 setting	Effect
CPU request	cpu.weight	CPU share under contention
CPU limit	cpu.max	Hard quota; the container is throttled when it uses the quota
Memory limit	memory.max	The container is OOM-killed if it exceeds the limit
Memory request	Used for scheduling and oom_score_adj	Not enforced as a cgroup limit by default

CPU limits cause *throttling*, not termination. A container that uses its quota early in a 100-millisecond period is stopped until the next period, which appears as latency spikes. `nr_throttled` in `cpu.stat` shows how often this occurs.

To inspect cgroups:

```
cat /proc/self/cgroup           # the cgroup path of the current process
cat /sys/fs/cgroup/memory.max
cat /sys/fs/cgroup/memory.current
cat /sys/fs/cgroup/memory.events
cat /sys/fs/cgroup/cpu.max
cat /sys/fs/cgroup/cpu.stat
systemd-cgls                   # the cgroup tree
systemd-cgtop                  # resource use per cgroup
```

3.3 From image to running container

When the scheduler assigns a Pod to a node, the kubelet on that node starts it through the Container Runtime Interface (CRI). With containerd and runc, the sequence is as follows:

1. The kubelet calls the CRI `RunPodSandbox` method. containerd creates the Pod sandbox: a small *pause* container that holds the Pod's network and IPC namespaces. The CNI plugin configures the network namespace (section 5.4).
2. For each container, the kubelet calls `PullImage` if needed, then `CreateContainer` and `StartContainer`.
3. containerd unpacks the image layers into snapshots and prepares an overlayfs root filesystem (section 4.2).
4. containerd writes an OCI *bundle*: the root filesystem and a `config.json` that specifies the command, environment, mounts, namespaces to create or join, cgroup path, device rules, capabilities, and seccomp profile.
5. A containerd shim invokes `runc create`. runc creates the new namespaces, joins the sandbox's network and IPC namespaces, places the process in its cgroup, sets up mounts, and calls `pivot_root` to switch to the container's root filesystem.
6. runc drops capabilities, applies the seccomp filter, and calls `execve` to start the container's entry point.

The containers in a Pod share the sandbox's network namespace, so they can reach each other on localhost and must not listen on the same port.

3.4 GPUs in containers

By default, a container has no access to GPU device files such as `/dev/nvidia0`, and its image does not contain the user-space driver libraries. The GPU kernel driver runs on the host, and the container needs two things from the host: the device nodes and user-space libraries that match the host driver version.

Three components cooperate to provide them:

1. **The NVIDIA device plugin** runs on each GPU node and advertises `nvidia.com/gpu` as an allocatable resource. After the scheduler places a Pod that requests GPUs, the kubelet calls the plugin's `Allocate` method, and the plugin returns the specific devices to assign, as environment variables, device paths, or Container Device Interface (CDI) device names.
2. **The NVIDIA Container Toolkit** is invoked by the runtime when the container is created. It mounts the driver libraries (`libcuda.so`, `libnvidia-ml.so`, and others) and the utilities such as `nvidia-smi` into the container, and it creates the device nodes for the assigned GPUs (`/dev/nvidia0`, `/dev/nvidiactl`, `/dev/nvidia-uvm`). With MIG, it exposes only the assigned GPU instance.
3. **The NVIDIA GPU Operator** installs and manages the driver, the toolkit, the device plugin, and monitoring components on each node.

Because the kernel driver is shared with the host, the CUDA version inside the image must be supported by the host's driver version. A newer driver supports older CUDA releases; an older driver may not support a newer CUDA release.

The device plugin, not the container runtime, decides which GPUs a container receives.

4. Filesystems and I/O

4.1 The VFS and file descriptors

The Virtual File System (VFS) is the kernel layer that gives every filesystem the same interface. ext4, XFS, NFS, overlayfs, and /proc all implement the same set of objects:

Object	Represents	Lifetime
super_block	One mounted filesystem	While the filesystem is mounted
inode	One file: owner, permissions, size, timestamps, data location	While the file exists or is in use
dentry	One name in a directory, linking the name to an inode	Cached for fast path lookup
file	One <i>open file description</i> : the current offset, access mode, and flags	From open until the last descriptor referring to it is closed

A file descriptor is an index into the process's descriptor table, and each entry points at a file object.

The relationships between these objects explain several behaviors:

- **An inode has no name.** Names belong to dentries. Renaming a file within a filesystem changes a dentry and leaves the inode unchanged.
- **A hard link is a second dentry for the same inode.** Inode numbers are unique only within one filesystem, so hard links cannot cross filesystems.
- **Two open calls create two file objects.** Each has its own offset, so two processes can read the same file independently.
- **fork and dup share one file object.** Parent and child share the offset, so a read in one process advances the position for the other.
- **A deleted file stays on disk while it is open.** Removing the last name (unlink) does not free the inode while a file object still refers to it. `df` shows the space as used, and `ls -l` lists the files involved.

The error convention

System calls report failure by returning -1 and setting `errno` to a code that identifies the cause. `errno` is thread-local, so threads do not overwrite each other's error codes. Listing 2.10 opens three paths and reports the result of each.

Listing 2.10 errno.c: how system calls report failure

```

/* errno.c: the syscall boundary reports failure with -1 plus errno. */
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

static void try_open(const char *path)
{
    char buf[64];
    ssize_t n;
    int fd = open(path, O_RDONLY);

    if (fd < 0) {
        printf("open(\"%s\") = -1 errno=%d (%s)\n", path, errno, strerror(errno));
        return;
    }
    printf("open(\"%s\") = %d\n", path, fd);

    n = read(fd, buf, sizeof buf - 1);    /* may return fewer bytes than asked */
    if (n > 0) {
        buf[n] = '\0';
        if (buf[n - 1] == '\n')
            buf[n - 1] = '\0';
        printf("  read() returned %zd byte(s): \"%s\"\n", n, buf);
    }
    close(fd);
}

int main(void)
{
    try_open("/definitely/not/here");
    try_open("/etc/hostname");
    try_open("/etc/shadow");    /* exists, but not readable by this uid */
    return 0;
}

```

```

$ gcc -O2 -o errno errno.c && ./errno
open("/definitely/not/here") = -1 errno=2 (No such file or directory)
open("/etc/hostname") = 3
  read() returned 8 byte(s): "yahboom"
open("/etc/shadow") = -1 errno=13 (Permission denied)

```

The output shows three details of the file interface:

- **Different failures have different codes.** ENOENT (2) means the path does not exist, and EACCES (13) means the caller lacks permission. A caller can handle the two cases differently, for example by creating a missing file but reporting a permission error.
- **Descriptors are small integers, allocated lowest first.** The first open returned 3 because 0, 1, and 2 are standard input, output, and error. The table has a per-process limit (`ulimit -n`), and a server that leaks descriptors eventually fails with EMFILE.

- **read can return fewer bytes than requested.** Here it returned 8 bytes, the size of the file, although the buffer held 63. A short read does not indicate an error or the end of the file; only a return value of 0 means end of file. Correct code calls `read` in a loop until it has the bytes it needs. Handling short reads and writes incorrectly is a common bug in systems code, especially with sockets and pipes.

4.2 Filesystems used on Kubernetes nodes

Filesystem	Type	Used for
ext4, XFS	Local block filesystems	Node root disk, container image storage, local volumes
overlayfs	Union filesystem	Container root filesystems built from image layers
tmpfs	Memory-backed	<code>emptyDir</code> with medium: Memory, <code>/dev/shm</code> , Secret volumes, projected service account tokens
NFS, CephFS	Network filesystems	Shared ReadWriteMany volumes
FUSE	User-space filesystems	Object storage mounts such as <code>gcsfuse</code> and <code>s3fs</code>
Block devices via CSI	Cloud disks	EBS, Persistent Disk, and other ReadWriteOnce volumes

Files written to `tmpfs` are memory, and they are charged to the Pod's memory cgroup. A Pod that writes large files to a memory-backed `emptyDir` can be OOM-killed.

overlayfs

`overlayfs` presents several directories as one filesystem. For a container, the *lower* directories are the read-only image layers, and the *upper* directory is a writable layer that belongs to the container.

- **Reading** a file returns it from the topmost layer that contains it.
- **Writing** to a file that exists only in a lower layer triggers a *copy-up*: the kernel copies the entire file to the upper layer before applying the write. The first write to a large file, such as a database file included in the image, can be slow for this reason.
- **Deleting** a lower-layer file creates a *whiteout* entry in the upper layer that hides the file. The image layer is unchanged, so the image does not shrink.

Write data that changes often to a volume rather than to the container's root filesystem.

4.3 Mount propagation

Each mount namespace has its own mount table. *Mount propagation* controls whether a mount created in one namespace appears in another. Each mount point has one of four propagation types:

Type	Behavior
shared	Mounts and unmounts propagate in both directions between peers
slave	Mounts propagate from the master into this mount, but not back
private	No propagation in either direction
unbindable	Private, and cannot be bind-mounted

Kubernetes exposes these types through the `mountPropagation` field on a volume mount:

mountPropagation	Propagation	Typical user
None (default)	private	Most containers
HostToContainer	rslave	Containers that must see mounts the host creates later
Bidirectional	rshared	CSI node plugins that mount volumes for other Pods; requires a privileged container

Incorrect propagation causes two common symptoms: a volume that a CSI driver mounted is not visible inside the application's container, or unmounting fails with `device or resource busy` because a copy of the mount remains in another namespace.

```
findmnt -o TARGET,PROPAGATION /
cat /proc/self/mountinfo
findmnt -R /var/lib/kubelet | head -50
mount | grep -E 'overlay|nvidia'
```

4.4 The I/O stack and pressure metrics

A write from an application passes through several layers:

```
application -> VFS -> filesystem -> page cache -> block layer -> device driver -> disk
```

A buffered `write` normally returns after the data is copied into the page cache. The pages are *dirty* until kernel writeback threads write them to the device. A program that needs data to be on disk before continuing calls `fsync` or `fdatasync`, which waits for writeback to complete and can be slow. When dirty pages accumulate beyond `vm.dirty_ratio`, the kernel makes writing processes wait.

Tools for storage performance:

Tool	What it shows
<code>iostat -x 1</code>	Per-device <code>r_await</code> and <code>w_await</code> (average latency in ms), <code>aqu-sz</code> (queue depth), and <code>%util</code>
<code>pidstat -d 1</code>	Read and write rates per process
<code>iotop</code>	Processes ordered by current I/O
<code>/proc/pressure/ io</code>	Pressure Stall Information (PSI) for I/O

On SSDs and NVMe devices, which serve many requests in parallel, `%util` can reach 100% while the device still has capacity. Latency (`await`) and pressure are more reliable indicators.

Pressure Stall Information

PSI reports the share of time that tasks were delayed waiting for a resource. The files `/proc/pressure/cpu`, `/proc/pressure/memory`, and `/proc/pressure/io` contain lines like the following:

```
some avg10=2.04 avg60=0.75 avg300=0.40 total=157622356
full avg10=0.00 avg60=0.13 avg300=0.08 total=51202355
```

- `some` is the percentage of time in which at least one task was stalled on the resource.
- `full` is the percentage of time in which all non-idle tasks were stalled at once, so no useful work was done.
- `avg10`, `avg60`, and `avg300` are averages over 10, 60, and 300 seconds.

Each cgroup has the same files (`cpu.pressure`, `memory.pressure`, `io.pressure`), so you can measure pressure for one container. Utilization shows how busy a resource is; pressure shows whether work is waiting for it.

5. Networking

5.1 Sockets and TCP states

A socket is a kernel object that an application uses as a communication endpoint. A TCP server and client use these calls:

Server	Client	Purpose
<code>socket()</code>	<code>socket()</code>	Create the socket
<code>bind()</code>	(optional)	Assign a local address and port
<code>listen(backlog)</code>		Mark the socket as accepting connections
<code>accept()</code>	<code>connect()</code>	Complete a connection; <code>accept</code> returns a new socket for it
<code>read()</code> / <code>write()</code>	<code>read()</code> / <code>write()</code>	Exchange data
<code>close()</code>	<code>close()</code>	Close the connection

After `listen`, the kernel completes the TCP handshake for incoming connections without the application's involvement and places them in the *accept queue*. The queue's size is the `backlog` argument, capped by `net.core.somaxconn`. If the application does not call `accept` quickly enough, the queue fills and new connections are dropped or reset. For a listening socket, `ss -ltn` shows the current queue length in Recv-Q and the maximum in Send-Q. `nstat -az TcpExtListenOverflows` counts overflows.

A TCP connection moves through a set of states. The ones you encounter most often when debugging are:

State	Side	Meaning
LISTEN	Server	Waiting for connections
SYN-SENT / SYN-RECV	Client / server	Handshake in progress
ESTABLISHED	Both	Connection open
CLOSE-WAIT	The side that received the first FIN	The peer has closed; this application has not yet called <code>close()</code>
TIME-WAIT	The side that closed first	Closed; kept for 60 seconds on Linux so that delayed packets are not delivered to a new connection

Two patterns are worth recognizing:

- **Many CLOSE-WAIT sockets** point to a bug in the local application. The remote side closed the connection, and the application never closed its socket. The count grows until the process runs out of file descriptors.

- **Many TIME-WAIT sockets** are normal on a host that opens many short outgoing connections. They become a problem only if the host runs out of ephemeral ports for a given destination. Connection pooling and keep-alive reduce the number.

```
ss -tulpn           # listening TCP and UDP sockets, with processes
ss -tan state established # established TCP connections
ss -tan state close-wait # connections waiting for the application to close
ss -tnp | grep :443   # connections to or from port 443
ss -s               # counts by state
```

5.2 The path of a packet through a host

Receiving. A packet arrives at the network interface card (NIC), which copies it with DMA into a receive ring buffer in memory and raises an interrupt. The driver processes the ring with NAPI polling, which handles many packets per interrupt under load. The packet then passes through these stages:

```
NIC ring buffer
-> XDP hook (earliest point; eBPF programs can drop or redirect here)
-> GRO (merge segments of the same flow)
-> tc ingress
-> netfilter PREROUTING (conntrack, DNAT)
-> routing decision
    -> local delivery: netfilter INPUT -> TCP/UDP -> socket receive queue -> application
    -> forwarding:    netfilter FORWARD -> POSTROUTING -> egress interface
```

Sending. Data written by an application is copied into the socket's send buffer, and TCP segments it. The packet then passes through routing, netfilter OUTPUT and POSTROUTING (where SNAT and masquerading occur), the queueing discipline (tc), and the driver's transmit ring.

Packets can be dropped at almost every stage. To find where, check `ethtool -S <nic>` for drops at the NIC, `/proc/net/softnet_stat` for drops in the kernel's receive processing, `nstat` for protocol-level counters such as retransmissions, and `conntrack -S` for connection tracking failures.

5.3 netfilter, conntrack, and kube-proxy

netfilter is the kernel framework that provides hooks (PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING) where packets can be filtered, modified, or translated. iptables and its successor nftables are the user-space tools that configure rules at those hooks. iptables organizes rules into tables (raw, mangle, nat, filter) and chains.

conntrack records the state of each connection that passes through the host. It serves two purposes. Stateful firewall rules can match on it, for example to allow packets that belong to an ESTABLISHED connection. And NAT depends on it: after the first packet of a connection is translated, conntrack applies the same translation to every later packet and reverses it for replies.

kube-proxy implements Kubernetes Services on each node. In iptables mode, it programs rules in the nat table that match a Service's cluster IP and port and apply DNAT to a randomly chosen backend Pod IP. conntrack translates the reply back, so the client sees the Service address. kube-proxy also supports ipvs and nftables modes. Some CNI plugins, such as Cilium, replace kube-proxy and implement Services with eBPF programs instead of netfilter rules.

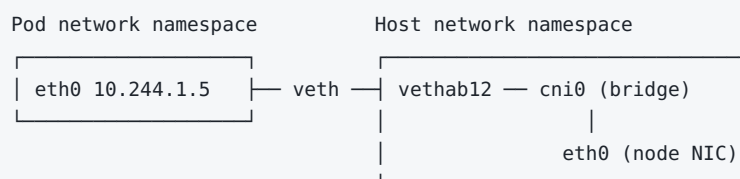
The conntrack table has a fixed maximum size. When it is full, the kernel drops new connections and logs `nf_conntrack: table full, dropping packet`. Compare the current and maximum values when a busy node drops connections intermittently:

```
sysctl net.netfilter.nf_conntrack_count net.netfilter.nf_conntrack_max
conntrack -S                # per-CPU statistics, including drops and insert failures
iptables -t nat -L -n -v | head # NAT rules, with packet counts
nft list ruleset | head
```

5.4 Network namespaces, veth pairs, and CNI

Each Pod has its own network namespace, with its own interfaces, IP address, routes, and ports. The Container Network Interface (CNI) plugin connects that namespace to the node's network. When the runtime creates a Pod sandbox, it calls the CNI plugin, which typically performs these steps:

1. Create a *veth pair*, two virtual interfaces connected like the two ends of a cable.
2. Move one end into the Pod's network namespace and name it `eth0`.
3. Attach the other end to the host network, for example to a bridge, or leave it as a routed interface.
4. Assign an IP address to `eth0` from the node's Pod CIDR, using the IPAM plugin.
5. Add routes, and in some plugins, network policy rules.
6. Return the interface and IP address to the runtime.



Traffic to a Pod on another node is routed or encapsulated (for example with VXLAN) by the plugin. Inside a Pod, `localhost` refers to the Pod's network namespace, not to the node. A process listening on `127.0.0.1` in one Pod cannot be reached from another Pod or from the node's `localhost`.

To inspect a Pod's network namespace from the node, find the PID of a process in the Pod and enter its namespace:

```
nsenter -t <pid> -n ip addr
nsenter -t <pid> -n ss -tan
```

5.5 DNS in a Pod

Pods resolve names through the cluster DNS service, usually CoreDNS. The kubelet writes `/etc/resolv.conf` in each container, similar to the following:

```
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

The search list lets a Pod resolve a Service by a short name, such as `api` or `api.other-namespace`. The `ndots:5` option controls when that list is used: a name with fewer than five dots is first tried with each search suffix appended, and only then as an absolute name.

As a result, a lookup of an external name such as `storage.example.com` (two dots) first queries `storage.example.com.default.svc.cluster.local`, then the other search domains, before it queries the real name. Each attempt can be sent for both A and AAAA records. This increases DNS load and latency for applications that connect to many external hosts. Two fixes are common:

- Write external names as fully qualified domain names with a trailing dot, such as `storage.example.com.`
- Lower `ndots` for the Pod with `dnsConfig.options`.

A Pod with `hostNetwork: true` uses the node's `/etc/resolv.conf` by default and cannot resolve cluster Service names unless its `dnsPolicy` is `ClusterFirstWithHostNet`.

6. Debugging an unhealthy node

6.1 A triage order

When a node or its workloads misbehave, check from the outside in. Each step either finds the problem or rules out a layer:

Step	Layer	Commands
1	Node status in the cluster	<code>kubectl get node</code> , <code>kubectl describe node</code> (conditions, taints, allocatable)
2	Basic resources	<code>uptime</code> , <code>free -h</code> , <code>df -h</code> , <code>df -i</code> , <code>top</code>
3	Kernel messages	<code>dmesg -T \</code> <code>tail -50</code> (OOM kills, disk errors, driver errors)
4	Container runtime	<code>systemctl status containerd</code> , <code>journalctl -u containerd</code> , <code>crictl ps -a</code>
5	kubelet	<code>systemctl status kubelet</code> , <code>journalctl -u kubelet</code>
6	Workload	<code>kubectl describe pod</code> , <code>kubectl logs</code> , <code>kubectl get events</code>
7	GPUs, on GPU nodes	<code>nvidia-smi</code> , device plugin logs, Xid errors in <code>dmesg</code>

`df -i` checks inode use. A filesystem can run out of inodes while it still has free space, for example when a workload creates millions of small files.

6.2 Scenario: Pods stuck in Pending

A Pod in Pending has not been placed on a node, or has been placed and its containers have not been created yet. Run `kubectl describe pod <pod>` first. The Events section usually states the reason, for example `0/12 nodes are available: 4 Insufficient nvidia.com/gpu, 8 node(s) had untolerated taint.`

Common causes:

- No node has enough allocatable CPU, memory, or extended resources such as GPUs.
- No node satisfies the Pod's nodeSelector, node affinity, or topology spread constraints.
- The nodes have taints that the Pod does not tolerate.
- A PersistentVolumeClaim is unbound, or its volume is in a zone that no eligible node is in.
- The GPU nodes do not advertise `nvidia.com/gpu` at all, because the device plugin is not running (section 6.5).
- The scheduler is not running. In this case the Pod has no scheduling events.

A ResourceQuota violation does not produce a Pending Pod. The API server rejects the Pod when it is created, so the error appears in the events of the Deployment's ReplicaSet or the Job, and no Pod exists.

```
kubectl describe pod <pod>
kubectl get events -n <namespace> --sort-by=.lastTimestamp
kubectl describe node <node> | grep -A8 -E 'Taints|Allocatable|Allocated resources'
kubectl get pvc -n <namespace>
```

6.3 Scenario: Pod stuck in ContainerCreating

The Pod has a node, and the kubelet is preparing it. The cause is on that node:

Symptom in events	Likely cause	Where to look
ErrImagePull, ImagePullBackOff	Wrong image name or tag, missing registry credentials, registry unreachable	kubectl describe pod, crictl pull
FailedMount, FailedAttachVolume	CSI driver problem, volume attached to another node	kubectl describe pod, journalctl -u kubelet, CSI node plugin logs
FailedCreatePodSandBox	CNI plugin error, IP address exhaustion	journalctl -u kubelet, journalctl -u containerd, CNI Pod logs
Device allocation errors	Device plugin cannot allocate the requested devices	Device plugin logs

6.4 Scenario: Pod in CrashLoopBackOff

The container starts and then exits repeatedly, and the kubelet waits longer between restarts each time. The container's exit code identifies how it ended. An exit code above 128 means the process was killed by a signal, and the signal number is the exit code minus 128.

Exit code	Meaning	Next step
1, or another small number	The application exited with an error	kubectl logs <pod> --previous
137	SIGKILL (128 + 9)	If the reason is OOMKilled, check the memory limit and memory.events; otherwise check for a failing liveness probe
139	SIGSEGV (128 + 11)	The application crashed; check logs and core dumps
143	SIGTERM (128 + 15)	The container was asked to stop, for example by a liveness probe failure or a deletion

kubectl logs --previous shows the output of the last terminated container, which is usually where the error message is. kubectl describe pod shows the last state, the exit code, and probe failures.

6.5 Scenario: a GPU node cannot allocate GPUs

Check each layer in order, from the driver up to the Pod:

1. Is the driver working on the host?

```
bash nvidia-smi
```

If this fails, the kernel module is not loaded, the driver installation failed, or the node needs a reboot after a driver update.

1. Is the device plugin running?

```
bash kubectl get pods -n gpu-operator -l app=nvidia-device-plugin-daemonset -o wide
kubectl logs -n gpu-operator -l app=nvidia-device-plugin-daemonset
```

1. Does the node advertise GPUs?

```
bash kubectl describe node <node> | grep -E 'nvidia.com/gpu'
```

If `nvidia.com/gpu` is missing from Capacity and Allocatable, the device plugin has not registered with the kubelet.

1. **Does the Pod request GPUs correctly?** `nvidia.com/gpu` is an extended resource. It must appear in limits; if it also appears in requests, the two values must be equal. Extended resources cannot be overcommitted or requested in fractions.

2. Does the container see the GPU?

```
bash kubectl exec <pod> -- nvidia-smi
```

If the command is missing or reports no devices, the container toolkit did not inject the libraries and devices. Check the runtime configuration that the GPU Operator installs.

1. Is there a hardware or driver fault?

```
bash dmesg -T | grep -iE 'xid|nvrn' nvidia-smi -q -d ECC nvidia-smi -q -d ROW_REMAPPER
# Ampere and newer nvidia-smi -q -d PAGE_RETIREMENT # older architectures
```

The driver logs *Xid* errors with a number that identifies the class of fault. For example, Xid 79 means the GPU has fallen off the PCIe bus, and Xid 48 reports an uncorrectable ECC error. Xid errors together with ECC errors indicate a hardware or driver problem rather than a Kubernetes configuration problem. Cordon the node and follow the hardware replacement process.

6.6 Scenario: high CPU use or load without an obvious cause

Start by determining whether the time is spent in user space, in the kernel, or waiting:

```
cat /proc/loadavg
top -H # per thread; compare %us, %sy, %wa, and %st in the header
pidstat -u 1 # CPU use per process over time
vmstat 1 # r = runnable tasks, b = tasks in state D
ps -eo pid,stat,wchan:32,comm | awk '$2 ~ /D/' # tasks in uninterruptible sleep
```

Interpret the results as follows:

- **High %sy (system time).** The kernel is busy on behalf of processes, for example with many system calls, lock contention, or memory reclaim. Use `perf top` to find the kernel functions, and `strace -c -p <pid>` to count system calls.
- **High %wa or many tasks in state D.** Tasks are waiting for I/O. The load average is high although the CPUs may be idle. Check `iostat -x` and `/proc/pressure/io`.
- **High %st (steal).** On a virtual machine, the hypervisor is giving the CPU to other guests.
- **One process with high %us.** Profile it. The `wchan` column shows the kernel function a sleeping task is waiting in.

To sample where a process spends CPU time, record stack traces:

```
perf record -F 99 -g -p <pid> -- sleep 10
perf report
```

If the node is a Kubernetes node, also check CPU throttling in the container's `cpu.stat`. A throttled container can be slow while the node's total CPU use is low.

7. Performance and observability tools

The table lists the main tools by the resource they examine. For a structured approach, check *utilization*, *saturation*, and *errors* for each resource in turn; Brendan Gregg calls this the USE method.

Resource	Tools	What to look for
CPU	top, mpstat -P ALL 1, pidstat -u 1, perf top	User versus system time, uneven use across CPUs, run-queue length
CPU in cgroups	cpu.stat, cpu.pressure	nr_throttled, pressure
Memory	free -h, vmstat 1, /proc/pressure/memory, memory.events	Available memory, swap activity, reclaim, OOM kills
Disk	iostat -x 1, pidstat -d 1, /proc/pressure/io	Latency, queue depth, pressure
Network	ss -s, nstat, sar -n DEV 1, ethtool -S, tcpdump	Retransmissions, drops, listen overflows
System calls	strace -c, perf trace	Frequent or slow calls, EAGAIN loops
Kernel tracing	bpftrace, perf, ftrace	Latency and events inside the kernel
Containers	crictrl stats, systemd-cgtop	Resource use per container

bpftrace examples

bpftrace attaches small eBPF programs to kernel events with little overhead. Each command runs until you press Ctrl+C:

```
# Print every program executed on the host
bpftrace -e 'tracepoint:sched:sched_process_exec { printf("%s -> %s\n", comm, str(args->filename)); }'

# Print every file opened, with the process name
bpftrace -e 'tracepoint:syscalls:sys_enter_openat { printf("%s %s\n", comm, str(args->filename)); }'

# Count system calls by process name
bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[comm] = count(); }'
```

When to use perf

Question	Command
Which functions use the most CPU right now?	<code>perf top</code>
Where does one process spend its CPU time?	<code>perf record -F 99 -g -p <pid> -- sleep 10</code> , then <code>perf report</code> or a flame graph
What are the hardware counters: instructions, cache misses, TLB misses?	<code>perf stat -p <pid></code>
Is there lock contention?	<code>perf lock record</code> and <code>perf lock report</code> (requires kernel support)

8. Containers compared with virtual machines

Aspect	Container	Virtual machine
Isolation mechanism	Namespaces, cgroups, capabilities, seccomp, LSMs	Hardware virtualization (for example KVM)
Kernel	Shared with the host	A separate guest kernel
Start time	Milliseconds, the time to start a process	Seconds, the time to boot a kernel
Attack surface	The host kernel's system call interface, reduced by seccomp	The hypervisor and its virtual devices
Devices	Device nodes exposed by the runtime	Emulated devices, paravirtual devices, or PCIe passthrough
GPU sharing options	Device plugin, time-slicing, MPS, MIG	PCIe passthrough, vGPU, MIG-backed vGPU

Cloud GPU platforms use both models. Kubernetes with the device plugin serves container workloads, and GPU virtual machines serve customers who need their own kernel or drivers. Sandboxed runtimes such as Kata Containers run each Pod inside a lightweight VM to combine the container interface with VM isolation.

9. Interview questions

What happens when you run `docker run nginx`?

1. The Docker CLI sends the request to the Docker daemon, which delegates to `containerd`.
2. If the image is not present, `containerd` pulls its layers and unpacks them into snapshots.
3. `containerd` prepares an overlayfs root filesystem and an OCI bundle with the runtime configuration.
4. `runc` creates namespaces, places the process in a cgroup, sets up mounts and `pivot_root`, applies capabilities and `seccomp`, and calls `execve` for the entry point.
5. The `nginx` master process runs as PID 1 in its PID namespace, with its own network namespace connected to the `docker0` bridge through a veth pair.

How does a memory limit terminate a container?

The kernel charges the container's anonymous memory and page cache to its cgroup. When `memory.current` reaches `memory.max`, the kernel reclaims memory charged to the cgroup. If reclaim cannot bring usage below the limit, the kernel runs the OOM killer for that cgroup only and kills a process in it with `SIGKILL`. The container exits with code 137, and Kubernetes reports `OOMKilled`.

A container runs `uname -r` and `ps aux`. What does it see?

`uname -r` prints the host's kernel version, because containers share the host kernel. `ps aux` shows only the processes in the container's PID namespace, starting with PID 1. A Pod with `hostPID: true` shares the host's PID namespace and sees every process on the node.

Why does `free` show almost no free memory on a healthy server?

Linux uses otherwise idle memory for the page cache and reclaims it when applications need memory. `free` is expected to be low. The `available` column estimates how much memory applications can use without swapping, and memory PSI shows whether tasks are waiting for memory.

What is the difference between a container and a virtual machine?

A container is a set of processes on the host kernel, isolated by namespaces and limited by cgroups. A virtual machine runs its own kernel on virtualized hardware. Containers start faster and use less memory. Virtual machines provide a stronger isolation boundary and can run a different kernel.

What is a zombie process, and how do you remove one?

A zombie is a process that has exited but whose parent has not called `wait` to read its exit status. You cannot kill a zombie, because it is not running. Either the parent reaps it, or the parent exits, the zombie is reparented to PID 1, and PID 1 reaps it. In a container, make sure PID 1 reaps orphans.

How do you debug a GPU node that cannot run GPU Pods?

Work from the hardware up: `nvidia-smi` on the host, the device plugin Pods and their logs, `nvidia.com/gpu` in the node's allocatable resources, the Pod's resource limits, `nvidia-smi` inside the container to test runtime injection, and finally Xid and ECC errors for hardware faults.

What is the page cache, and should you drop it?

The page cache holds file data in memory. Reads served from it avoid disk I/O, and buffered writes go to it before they are written to disk. Dropping it with `echo 3 > /proc/sys/vm/drop_caches` is rarely useful in production: it frees memory the kernel would have reclaimed anyway, and later reads must go to disk again. It is mainly useful for making cold-cache benchmarks repeatable.

What is conntrack, and how can it cause connection failures?

conntrack is the kernel's connection tracking table. Stateful firewall rules and NAT, including kube-proxy's Service DNAT, depend on it. The table has a maximum size. On a node with many short connections, the table can fill, and the kernel then drops new connections and logs `nf_conntrack: table full`. Compare `nf_conntrack_count` with `nf_conntrack_max`, and raise the limit or reduce connection churn.

Summary

- Linux represents both processes and threads as tasks. A process is a thread group, and clone flags determine what a new task shares.
- fork uses copy-on-write, so only pages that are written are copied. exec replaces the program and keeps the process ID and open descriptors that are not marked close-on-exec.
- Every system call has a fixed cost for entering and leaving the kernel. Batching, buffering, and shared-memory interfaces reduce the number of crossings.
- Virtual memory is allocated lazily through page faults. VSZ, RSS, and PSS measure different things, and a successful malloc does not guarantee physical memory.
- glibc serves small allocations from a brk heap and large ones from mmap, and freed heap memory often remains in the process.
- A container is a process tree restricted by namespaces, which control visibility, and cgroups, which control resource use. A memory limit that is exceeded causes an OOM kill inside that cgroup only.
- The VFS separates names (dentries), files (inodes), and open files (file objects). A read can return fewer bytes than requested.
- Kubernetes networking relies on network namespaces, veth pairs, netfilter or eBPF, and conntrack. The ndots:5 default affects DNS load for external names.
- Debug a node from the outside in: node status, resources, kernel messages, runtime, kubelet, workload, and GPU layer.

Further reading

Source	Use it for
Remzi and Andrea Arpaci-Dusseau, <i>Operating Systems: Three Easy Pieces</i> (free at ostep.org)	A clear first introduction to virtual memory, scheduling, and concurrency
Randal Bryant and David O'Hallaron, <i>Computer Systems: A Programmer's Perspective</i>	How C code relates to machine code; chapters 8 and 9 cover processes and virtual memory
Michael Kerrisk, <i>The Linux Programming Interface</i>	A complete reference for Linux system calls, their errors, and their edge cases
W. Richard Stevens and Stephen Rago, <i>Advanced Programming in the UNIX Environment</i>	Signals, process control, and I/O
Robert Love, <i>Linux Kernel Development</i>	An introduction to the kernel's internal data structures
Brendan Gregg, <i>Systems Performance</i> , 2nd edition	Performance analysis methods and the tools in section 7
Ulrich Drepper, "What Every Programmer Should Know About Memory" (2007)	CPU caches, the TLB, and NUMA
The Linux kernel documentation, docs.kernel.org	cgroup v2 (admin-guide/cgroup-v2), PSI (accounting/psi), and overlayfs (filesystems/overlayfs)
Kubernetes documentation, kubernetes.io/docs	Pod QoS classes, resource management, DNS for Services and Pods

Source: [02-linux-concepts.md](#) · [PDF](#)